

SQL databases, joining and retrieving data. Portable data formats: csv, json.

1. Introduction to Relational Databases

Relational databases are the most common way business data is stored and organized. A relational database consists of **tables** that store data in a structured format with rows and columns.

- **Table:** A table in a relational database consists of records (rows) and fields (columns).
- **Primary Key:** A unique identifier for a record in a table.
- **Foreign Key:** A field in one table that links to the primary key in another table, creating a relationship between tables.
- **Relational Model:** It enforces rules about how data can relate across tables (e.g., foreign key relationships).

Popular Relational Database Systems:

- MySQL
- PostgreSQL
- SQLite
- SQL Server

SQL (Structured Query Language):

SQL is the standard language used to interact with relational databases. It allows you to query, insert, update, and delete data.

Common SQL Operations:

- **SELECT:** Retrieve data from the database.
- **INSERT INTO:** Add new records to a table.
- **UPDATE:** Modify existing data.
- **DELETE:** Remove records.

SQL Data Types:

- **Text Types:** VARCHAR, CHAR, TEXT
- **Numeric Types:** INT, FLOAT, DECIMAL
- **Date/Time Types:** DATE, TIME, TIMESTAMP

2. Joining and Retrieving Data

When data is spread across multiple tables, **joins** are used to combine this data into a meaningful result.

Types of Joins:

- **INNER JOIN:** Retrieves records with matching values in both tables.
- **LEFT JOIN** (or LEFT OUTER JOIN): Retrieves all records from the left table, and matching records from the right table. Returns NULL where there is no match.
- **RIGHT JOIN** (or RIGHT OUTER JOIN): Retrieves all records from the right table, and matching records from the left table.
- **FULL OUTER JOIN:** Retrieves all records where there is a match in one of the tables, filling NULLs where there are no matches.

Example

```
SELECT customers.name, orders.order_id  
  
FROM customers  
  
INNER JOIN orders ON customers.customer_id = orders.customer_id;
```

This query retrieves the customer name and order ID for customers with matching records in the orders table.

Filtering Data:

You can apply filters to your SQL queries using the **WHERE** clause:

```
SELECT * FROM orders WHERE order_date > '2023-01-01';
```

Sorting and Limiting Results:

- **ORDER BY:** Sorts the results based on a column.
- **LIMIT:** Restricts the number of rows returned.

```
SELECT * FROM orders ORDER BY order_date DESC LIMIT 10;
```

3. Portable Data Formats: CSV and JSON

When moving data between systems or storing it in a lightweight, portable way, we often use formats like **CSV** (Comma-Separated Values) and **JSON** (JavaScript Object Notation).

3.1. CSV (Comma-Separated Values):

- **Structure:** Data is stored in plain text, where each row is a line of text, and columns are separated by commas.
- **Usage:** Commonly used in spreadsheets (like Excel) and is widely supported for importing/exporting data.

Example:

```
id, name, age
1, John Doe, 30
2, Jane Smith, 25
```

Python Code to Read CSV:

```
import csv

with open('data.csv', newline='') as csvfile:

    reader = csv.DictReader(csvfile)

    for row in reader:

        print(row['name'], row['age'])
```

3.2. JSON (JavaScript Object Notation):

- **Structure:** Data is stored as key-value pairs, making it easier to read and write programmatically.
- **Usage:** Common in web applications, APIs, and data interchange between systems.

Example:

```
{
  "id": 1,
  "name": "John Doe",
```

```
"age": 30
}
```

Python Code to Read JSON:

```
import json

with open('data.json') as json_file:

    data = json.load(json_file)

    print(data['name'])
```

4. APIs and Automating Data Retrieval

APIs (Application Programming Interfaces) allow software to communicate with other systems. In the context of data retrieval, APIs often provide programmatic access to web-based services, like weather data, social media platforms, or financial data feeds.

REST APIs:

- **GET:** Retrieve data from a server.
- **POST:** Send data to a server.

Example: Using the Twitter API

To access Twitter data, you would first need API credentials. Using Python's requests library, you can interact with the Twitter API:

```
import requests

url = "https://api.twitter.com/2/tweets?ids=1441162451777609730"

headers = {"Authorization": "Bearer YOUR_ACCESS_TOKEN"}

response = requests.get(url, headers=headers)

data = response.json()

print(data)
```

5. Integrating Data in Python

Reading Data from Relational Databases in Python:

To interact with SQL databases, Python provides libraries like:

- **SQLite** (built into Python)
- **MySQL Connector** (for MySQL databases)
- **Psycopg2** (for PostgreSQL)

Example with SQLite:

```
import sqlite3

# Connect to database

conn = sqlite3.connect('database.db')

cursor = conn.cursor()

# Retrieve data

cursor.execute("SELECT * FROM customers")

rows = cursor.fetchall()

# Close connection

conn.close()
```

6. Summary

- **SQL** is used to access data stored in relational databases.
- **Joins** allow you to combine data across multiple tables.
- Data can be transferred in **CSV** or **JSON** formats.
- **APIs** provide a programmatic way to automate data retrieval.
- Python has built-in libraries for working with databases, CSV, JSON, and APIs.